

COMPUTATIONEEL DENKEN

Voornaam: _____

Achternaam: _____

Klas: _____



Sint-Lievenscollege



The future is now ...

Het begon zo'n slordige 70 jaar geleden. Wetenschappers ontwikkelden een nieuwe wereld. Een digitale wereld! Een wereld van enkele grote computersystemen die met elkaar in verbinding stonden. Computers die een gehele kamer vulden en code die stukje per stukje met de hand werd geschreven. Draai de klok zeventig jaar vooruit en we bevinden ons middenin de digitale revolutie! Onze samenleving wordt steeds meer beïnvloed en gestuurd door computersystemen, apps en algoritmes. Nagenoeg iedereen heeft een krachtig device in de broekzak zitten of om de pols. Talloze problemen, nu en in de toekomst, worden opgelost via die krachtige devices! Maar als jij wil meewerken aan die toekomst, zelf oplossingen uitwerken en jouw steentje bijdragen aan die digitale revolutie, moet je eerst leren **'computationeel denken'**!



... and the future needs you!

'Computationeel denken' is een manier van probleemoplossend denken. Een aanpak om problemen op te lossen door het goed gebruik van talloze digitale methodes die ons omringen. Dus door te leren om computers in te zetten als werkmiddelen of 'tools'.

Maar voordat je een computer kan gebruiken als hulpmiddel, moet je begrijpen hoe je dat kan doen. Leren denken zoals een computer dat doet. Een computer, groot of klein, is een strikt logische machine die werkt met heel duidelijke instructies en acties. Vriendelijk vragen aan een computer om een probleem op te lossen zal je niet ver brengen. Dat zal de computer niet begrijpen. Maar jouw probleem opdelen in korte acties en deze vertalen naar concrete stappen of code ... bingo!

Allemaal kleine programmeurs?

Wat willen we niet bereiken: van alle leerlingen kleine programmeurs maken. Programmeren is een prachtige stiel. Je ontwerpt apps en algoritmes die de wereld rondom ons fundamenteel bepalen. Hoe we leven, eten en zelfs daten! Programmeurs zijn de architecten van ons leven en onze toekomst, maar niet iedereen moet architect worden.

Wat willen we wél bereiken: dat je begrijpt hoe die apps algoritmes tot stand komen. Dat je kan denken zoals een computer en zo begrijpt hoe onze digitale wereld in elkaar steekt. De digitale toekomst behoort niet enkel toe aan de programmeurs, maar om een vinger in de pap te hebben, moet je wel enkele kneepjes en inzichten onder de knie hebben. Dat computationeel denken meer is dan code op een scherm tikt, bewijzen volgende mensen:



Timnit Gebru is een Amerikaanse computerwetenschapper en gespecialiseerd in AI. Haar focus ligt op vooroordelen en discriminatie die sluipen in onze computer-algoritmes. Vooroordelen die vrouwen, LGBTQIA+ en 'people of color' treffen.



Shoshana Zuboff is een Amerikaanse filosofe en professor die zich richt op hoe algoritmes onze toekomst bepalen. Meer nog: haar werk gaat specifiek over hoe grote databedrijven ons gedrag letterlijk sturen!

Inhoudsopgave



1) Wat is Computatoneel denken?

- o Decompositie - (pagina 4)
- o Patroonherkenning - (pagina 5)
- o Abstraheren - (pagina 6)
- o Algoritme ontwerpen - (pagina 7)
- o Debuggen - (pagina 8)

2) Een algoritme is: - (pagina 7)

- o Is schematisch
- o Is functioneel
- o Is leesbaar
- o Bevat programmeerconcepten
- o Bevat wiskundeconcepten
- o Bevat creativiteit

3) Programmeerconcepten leren zoals:

- o Sequentie - (pagina 9)
- o Selectie - (pagina 10)
- o Begrensde herhaling - (pagina 11)
- o Voorwaardelijke herhaling - (pagina 12)

4) Herhaling - (pagina 14)

5) Evaluatie! - (pagina 15 en 16)

Les- en leerplandoelen



De leerstof die we in deze syllabus verwerken is verbonden met de doelen basisgeletterdheid in de eerste graad van het secundair onderwijs. Deze doelen vindt de Vlaamse Gemeenschap uiterst belangrijk om door te geven aan de jeugd. Zo belangrijk, dat iedereen ze moet behalen. Wanneer we kijken naar het leerplan van het Katholiek Onderwijs Vlaanderen vinden we:

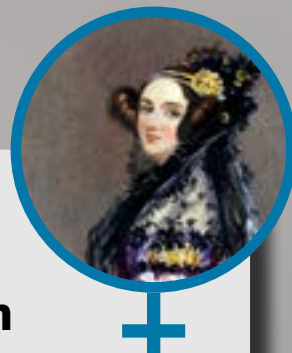
Basisgeletterdheid 17:

De leerling past een aangereikt algoritme toe om een probleem digitaal (plugged) en niet-digitaal (unplugged) op te lossen.

Computatoneel denken:

Decompositie, patroonherkenning, abstractie, algoritme maken, debuggen (testen en bijsturen) [Gemeenschappelijk leerplan ICT, LPD 3].

De eerste programmeur: Ada Lovelace Byron



Wanneer we zeggen dat de digitale revolutie zo'n zeventig jaar geleden begon, klopt dit niet volledig. Het ontwikkelen van de digitale wereld kon enkel maar gebeuren dankzij het werk van eerdere slimme wetenschappers. Het eerste "computerprogramma" werd niet geschreven in de twintigste eeuw, maar in 1843! Waar de computer- en wiskundewetenschappen nu vooral bevolkt worden door mannen, werd dat eerste computerprogramma geschreven door een vrouw genaamd Ada Lovelace. Ada was een wiskundige en ging een hele andere richting uit met haar leven dan dat haar vader (dichter Lord Byron) deed. Zo werkte ze samen met Charles Babbage aan de 'Analytische Machine'. Waar Charles de machine enkel zag als een wiskundetool, zag Ada het echte potentieel. In haar artikels voorzag ze dat zo'n analytische machine gebruikt zou kunnen worden om muziek te componeren, afbeeldingen te maken en wetenschap te bedrijven. In Ada's toekomst-droom leef jij nu!



Deze opdrachten zijn '**unplugged**' en voer je uit **zonder** computer.

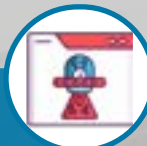


Deze opdrachten zijn '**plugged**' en voer je uit **mét** computer.



Scan de QR-code en krijg zo een **lesvideo** met een woordje uitleg.

CD - Stap 1: Decompositie



Wat voor ons simpel lijkt, is heel ingewikkeld voor een computer. Dit komt omdat wij heel goed zijn in compositie. Een compositie is een samenstelling van bv. handelingen of acties. We staan er niet meer bij stil. 'Fietsen' lijkt voor ons eenvoudig, maar een computer kent die vaardigheid niet. Bij een **decompositie** wordt een **complex probleem opgedeeld in kleinere deelproblemen of stapjes**. Die deelproblemen zijn veel meer behapbaar voor een computer en lossen we zo een voor een op.

Opgave 1: boterham smeren of tanden poetsen

Onze taal kent veel begrippen die een hele verzameling aan acties en handelingen bevatten. Begrippen die jij, als mens, wel onmiddellijk kan omzetten in een gewenst resultaat, maar die veel te ingewikkeld zijn voor een computer. Computers zijn immers strikt logische machines en kunnen heel moeilijk overweg met iets complex zoals taal.

Voor deze opgave heb je twee deelnemers nodig.

- o Kies een van de twee scenario's (boterham smeren of tanden poetsen);
- o Deelnemer 1: speelt de computer en voert de instructies letterlijk uit;
- o Deelnemer 2: geeft mondelinge instructies aan deelnemer 1.



Opgave 2 - Klasgesprek: waar loopt dit verkeerd?

De computer heeft het moeilijk met bovenstaande opdrachten, maar hoe komt dit? Leg kort uit waarom het verkeerd loopt bij de computer:

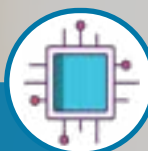
Leg kort uit wat we moeten aanpassen zodat de computer dit wel kan uitvoeren. Gebruik het begrip '**decompositie**' in jouw antwoord.

Opgave 3: dobbelsteen gooien

Voer de opdracht met de twee deelnemers nog eenmaal uit. Het scenario: je speelt een gezelschapsspel waarbij een dobbelsteen wordt gebruikt. De computer is aan de beurt en moet rollen met de dobbelsteen. Gebruik 'decompositie' en geef de gepaste instructies.



CD - Stap 2: Patroonherkenning



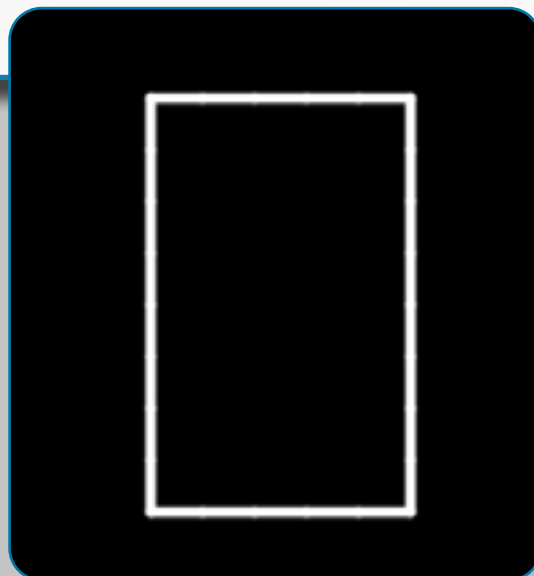
Wanneer we complexe zaken ontleden door **decompositie**, vinden we vaak **patronen** terug. Je vindt ze ook terug in het dagelijkse leven. Tegels op de vloer, de wandeling naar school, een patroon in een kledingstuk, de blaadjes aan de boom of de voortplanting van konijnen ... Patronen opsporen en begrijpen kan ons helpen om onze oplossing efficiënter te maken! Zo moet je niet telkens hetzelfde uitvinden!



Opgave: Olifant in de savanne

Een olifant in de savanne is uitgerust met een tracker. Deze geeft de locatie van de olifant door aan GPS-satellieten. Wanneer we dit patroon uittekenen, kunnen we een drone de olifantkudde laten volgen. Bekijk de voorgestelde code (1) en het wandelparkour (2) van de kudde hieronder. Ga op zoek naar een patroon en verbeter de code.

```
aantal vooruit -> 25
aantal vooruit -> 25
aantal vooruit -> 25
rechts draaien, 90
aantal vooruit -> 25
aantal vooruit -> 25
aantal vooruit -> 25
aantal vooruit -> 25
aantal vooruit -> 25
rechts draaien, 90
aantal vooruit -> 25
aantal vooruit -> 25
aantal vooruit -> 25
aantal vooruit -> 25
aantal vooruit -> 25
aantal vooruit -> 25
aantal vooruit -> 25
rechts draaien, 90
aantal vooruit -> 25
aantal vooruit -> 25
aantal vooruit -> 25
aantal vooruit -> 25
aantal vooruit -> 25
rechts draaien, 90
aantal vooruit -> 25
aantal vooruit -> 25
aantal vooruit -> 25
aantal vooruit -> 25
aantal vooruit -> 25
```



Waarom is deze code / oplossing niet goed?

Hoe zou jij de oplossing verbeteren?

Markeer wat best past:

Mijn verbeterde code of oplossing is beter voor de mens / de computer.



o dubbel van de som van 2 en X:





CD - Stap 4: Algoritme ontwerpen

Een **algoritme** is een **stappenplan** dat ons naar de juiste oplossing brengt. Je vindt ze bij een LEGO-bouwdoos of een IKEA-meubel. Bij het stappenplan is het belangrijk dat je de **instructies in de juiste volgorde, een voor een, uitvoert!** De volgorde wijzigen zal eindigen in een heel vreemde IKEA-kast ... Een goed algoritme voldoet aan volgende kenmerken:

- 1) Schematisch
- 2) Functioneel
- 3) Leesbaar
- 4) Programmeerconcepten
- 5) Wiskundeconcepten
- 6) Creativiteit

We focussen ons op de eerste drie in dit hoofdstuk.

1) Schematisch

Een algoritme is géén volledig geschreven boek, maar een overzichtelijk schema van de stappen die een persoon of een computer moet ondernemen.

2) Functioneel

Wanneer je de stappen nauwgezet volgt, moet je de oplossing bereiken. Het stappenplan moet dus werken of functioneel zijn.

3) Leesbaar

Een algoritme is zoals een boekje bij een LEGO- of IKEA-set. Dat wordt niet alleen gelezen door de ontwerper, maar moet leesbaar zijn voor iedereen. Ook onbekenden!

Opgave: de langste naam van de klas

Klasactiviteit: er worden twee speciale deelnemers aangeduid in de klasgroep.

- o Deelnemer 1: zal een algoritme schematisch noteren aan het bord.
- o Deelnemer 2: zal de instructies uitvoeren en de leerlingen in de klas bevragen.

Opgave: we gaan op zoek naar de leerling met de langste naam (voornaam + familienaam). Noteer de instructies **schematisch** op het klasbord en hieronder. **Let op de drie kenmerken!**



Stap 1:
Vraag naam

Stap 2:
Tel karakters

CD - Stap 5: Debugging

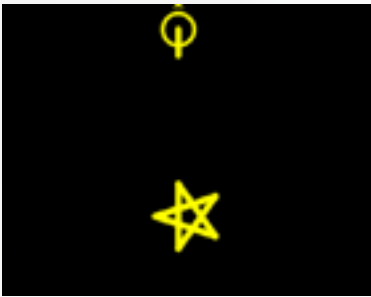


Een goed ontworpen algoritme zal ons tot de oplossing leiden ... maar wat als dat niet het geval is? Wat als onze computer of robot niet doet wat we vragen? Vaak spreken we dan van een **'bug'**. Dat is een fout in de code of in het ontwerp van onze oplossing. **Het opmerken van deze fouten, ze herkennen en de fout uit de oplossing verwijderen noemen we 'debuggen'**. De term hebben we te danken aan **Grace Hopper** toen in 1946 een mot, die vast zat in de bekabeling, haar PC in de war stuurde. De mot werd verwijderd en zorgvuldig in een boekje bewaard. In dat boekje maakte ze nota van wat er precies fout liep. Dit naslagwerkje is dus het allereerste **'bug report'**!

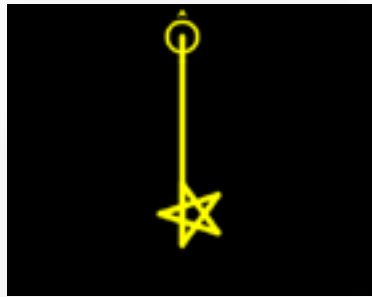
Gegeven 1: 'Debug' het schema

Mijn computerprogramma voor een printer / tekenrobot moet een ster tekenen met daarboven een lijntje. De kleur van de inkt moet geel zijn.

Gewenst resultaat:



Mijn resultaat:



Mijn algoritme:



Opgave 1: '

Duid met kleur aan waar de fout of 'bug' in mijn algoritme zich bevindt. Wat moet ik veranderen?

Opgave 2: Debug de code!

Het debuggen van echte computercode is niet moeilijker dan het debuggen van blokjescode zoals hierboven. De code hieronder geeft klanten die tien of meer cupcakes kopen 10% korting. Kopen ze er twintig of meer, krijgen ze 20% korting. Een cupcake kost 2 euro.

Gegeven:

o Klant 1 koopt 17 cupcakes;
o Klant 2 koopt 21 cupcakes.
`aantal_cupcakes = ...`
`prijs_per_cupcake = 2`
`prijs = (aantal_cupcakes * prijs_per_cupcake)`
ALS `aantal_cupcakes` gelijk aan 10:
 `totaalprijs = prijs / 100 * 110`
ALS `aantal_cupcakes` gelijk aan 20:
 `totaalprijs = prijs / 100 * 120`
`print(totaalprijs)`

Gevraagd:

- 1) Duid met kleur de hoofdzaken aan.
- 2) Duid met een ander kleur de fouten of bugs aan in het algoritme.
- 3) Noteer de correcte code hiernaast.

Correcte code:



Programmeerconcept 1: Sequentie

Computers zijn strikt logische machines. Wanneer we een algoritme ontwerpen met een schema, met blokjes of met codetaal, zullen ze onze instructies stap voor stap, in sequentie of volgorde, uitvoeren. Ze slaan dus niet zomaar een stap over, maar je kan ze wel pauzeren!

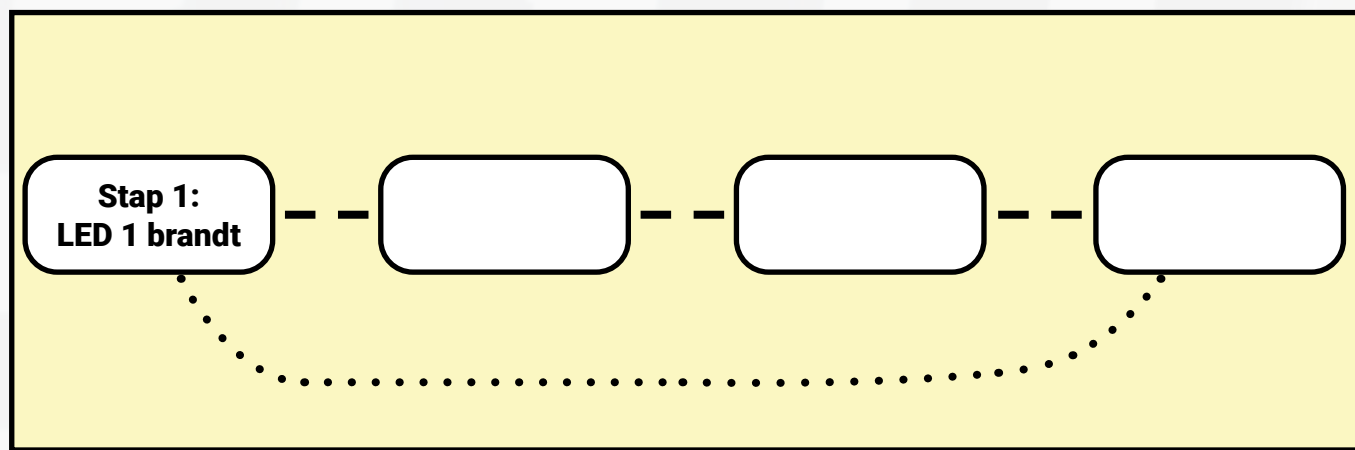


Opgave 1: knipperlicht aan de gevaarlijke oversteekplaats

Aan de Dampoort zijn er een aantal gevaarlijke punten voor fietsers en voetgangers. Dagelijks bezeren 14 kinderen zich in het verkeer van en naar school. Daarom besluit de stad Gent om het knooppunt te vernieuwen en een aantal lichten te plaatsen die aankomende auto's en vrachtwagens moeten waarschuwen. Hiervoor willen ze een verkeerslicht plaatsen met één lamp. Deze lamp moet knipperen, aan en uit. De LED moet 1 seconde aan staan en 1 seconde uit.

Stap 1: Duid met kleur de hoofdzaken aan in de opgave.

Stap 2: Maak een kort schema (tekening) van wat gevraagd wordt.
Werk stap voor stap, want zo denkt een computer ook!



Opgave 2: DwenguinoBlockly!

Stap 3: Open [DwenguinoBlockly](#) en maak jouw oplossing met de blokken in het deeltje 'Herhaal'.

Stap 4: Verplaats alle blokken in de 'herhaal' naar de 'zet klaar'. Voer het programma opnieuw uit. Denk na over wat het verschil is tussen code in de 'zet klaar' en code in de 'herhaal'. Waar voor kan dit verschil nuttig zijn?

Hoeveel keer wordt de code bij 'Zet Klaar' uitgevoerd? _____

Hoeveel keer wordt de code bij 'Herhaal' uitgevoerd? _____

Welke optie zal stad Gent kiezen voor die nieuwe verkeerslichten? Waarom?

Stap 5: Pas de waarde in het wacht-blok aan. Verhoog de waarde naar 10000 of verlaag het naar 100 milliseconden. Wat is het effect? Wat doet de LED?



Programmeerconcept 2: Selectie

Wanneer de computer een stuk code uitvoert, krijgen we steeds een oplossing. Maar we kunnen de code zodanig ontwerpen dat **meerdere uitkomsten mogelijk zijn!** Hiervoor gebruiken we **voorwaarden**. Aan de hand van de **voorwaarden** kan de computer **selecteren** welke stap er gezet moet worden. Daarvoor gebruiken we een **IF-ELSE-functie**. **IF** of **ALS** aan de voorwaarde wordt voldaan gebeurt het ene, **ELSE** of **ANDERS** gebeurt het andere.



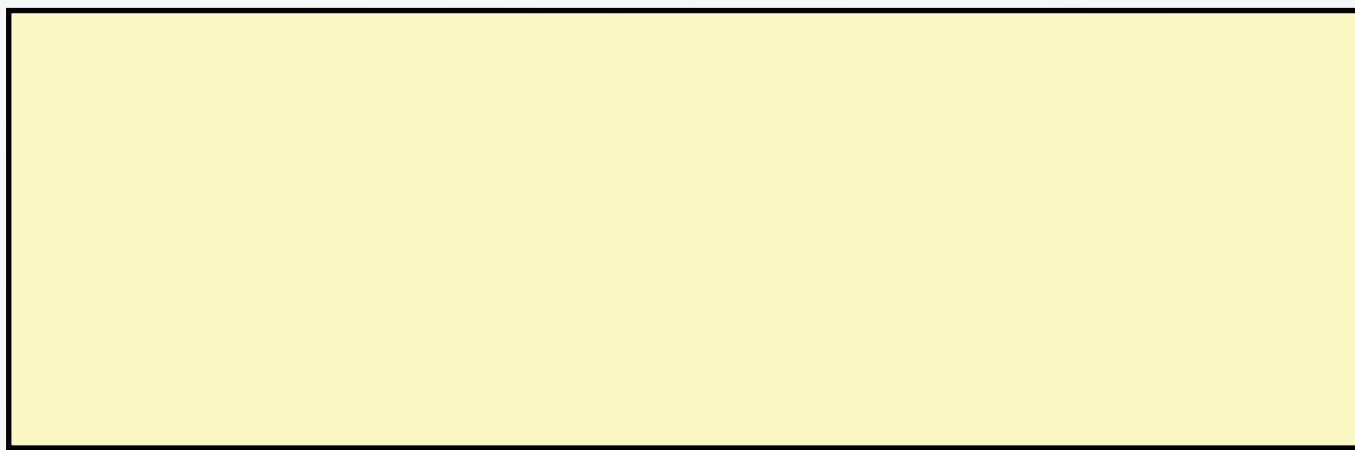
Opgave 1: Solden, korting vanaf 2 stuks

Tijdens de soldenperiode zie je vaak korting op kledij of andere artikelen. Soms krijg je die korting pas vanaf x aantal stuks. In een winkel krijg je 50% korting **ALS** je twee stuks koopt. **ANDERS** betaal je volle pot.

Stap 1: Duid met kleur de hoofdzaken aan in de opgave.

Stap 2: Maak een kort schema (tekening) van wat gevraagd wordt.

Werk stap voor stap, want zo denkt een computer ook!



Opgave 2: DwenguinoBlockly!

Stap 3: Open [DwenguinoBlockly](#) en maak jouw oplossing met de blokken in het deeltje 'zet klaar'. Gebruik een voorwaarde en ALS- en ANDERS-functie.

Stap 4: Download jouw oplossing en dien het in via de uploadzone van het vak!



Programmeerconcept 3: Begrensd Herhaling



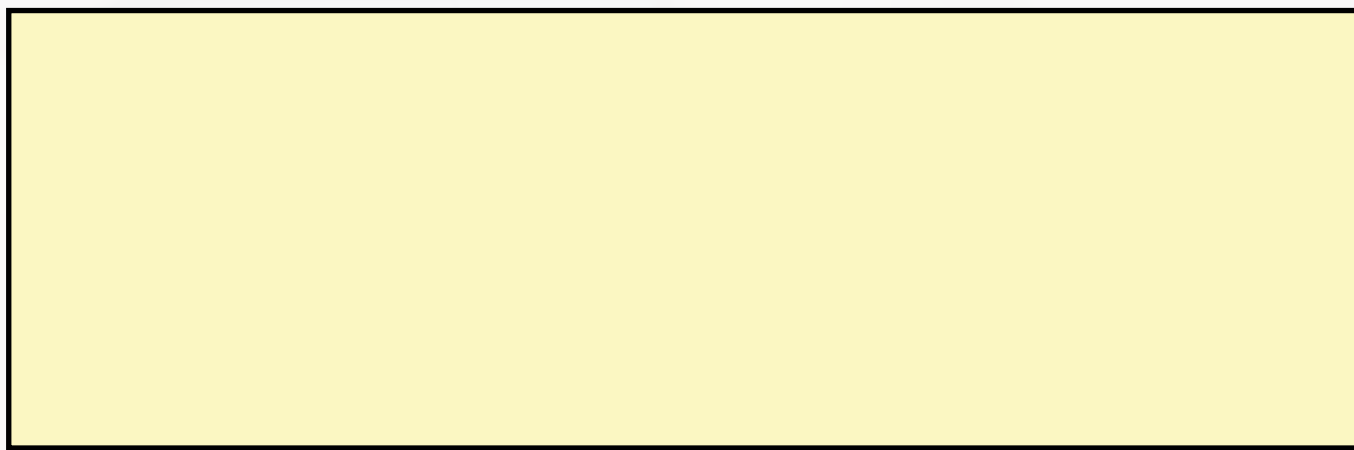
Bij de begrensd herhaling herkennen en maken we gebruik van een **patroon** in de code. Bij de **begrensd herhaling weten we exact hoeveel keer bepaalde acties herhaald moeten worden**. Niet meer, niet minder. Dit doen we aan de hand van een **'for-functie'** of **'een teller'**.

Opgave 1: Robotstofzuiger programmeren

In het werkatelier bij de stofzuigerfabrikant ontwikkelt men een zelfrijdende, volledig automatische stofzuiger. Men wil die ontwikkelen in stappen. Tijdens de eerste stap moet de stofzuiger starten, rechtdoor rijden en vanzelf stoppen na 2 seconden. Tijdens de tweede stap wil men dat de stofzuiger start, rechtdoor rijdt, draait ... en zo in een vierkant rijdt. Beide programma's worden na elkaar uitgevoerd. Er zit een pauze van 4 seconden tussen beide programma's.

Stap 1: Duid met kleur de hoofdzaken aan in de opgave.

Stap 2: Maak een kort schema (tekening) van wat gevraagd wordt.
Werk stap voor stap, want zo denkt een computer ook!



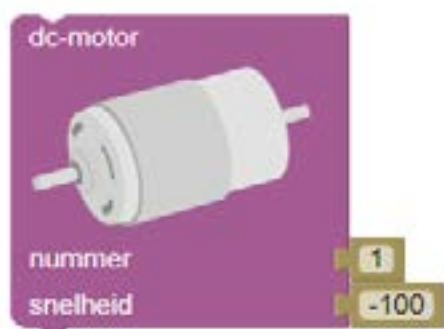
Opgave 2: DwenguinoBlockly!

Stap 3: Open [DwenguinoBlockly](#) en maak jouw oplossingen met de blokken in het deeltje *'Zet Klaar'*. Gebruik voor deze oefening de **'teller'**.

Stap 4: Download jouw oplossing en dien het in via de uploadzone van het vak!

Belangrijk: Hoe werken de motoren in DwenguinoBlockly?

In DwenguinoBlockly heeft de robot twee wielen die elk aangedreven worden door één motor. Er steekt dus één motor aan de linkerkant, één motor aan de rechterkant. Een motor heeft twee parameters die je kan aanpassen.



```
// bepaalt welke motor draait (1 of 2)  
// bepaalt de snelheid en richting!  
// -255 (achteruit) tot 255 (vooruit)
```


Programmeerconcept 4: Voorwaardelijke Herhaling



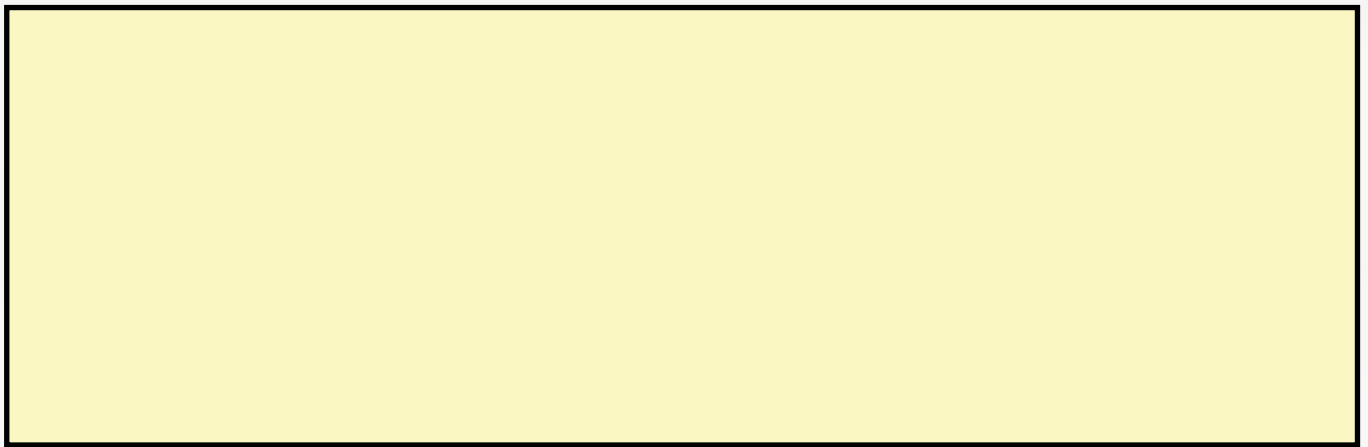
Bij de **voorwaardelijke herhaling** weten we **niet exact hoeveel keer een bepaalde set aan acties herhaald moet worden**. We weten alleen wanneer de computer mag stoppen, namelijk wanneer aan een voorwaarde is voldaan. Zo niet, blijft de herhaling doorlopen. Dit doen we aan de hand van een **zolang-functie** en een **voorwaarde**.

Opgave 1: Robotstofzuiger versie 2.0

Een zelfrijdende stofzuiger programmeren zoals we deden bij **Programmeerconcept 3**, dat zal in de praktijk niet goed werken. Als fabrikant weet je niet hoe groot elke kamer precies is. Daarom rusten we onze robot uit met sensoren zodat de robot zelf kan beslissen wanneer hij moet draaien. Bijvoorbeeld zodat hij niet hard tegen een muur 100 cm verderop aan knalt. Hier-voor gebruiken we een 'sonar' die afstanden kan meten via een geluidsgolf.

Stap 1: Duid met kleur de hoofdzaken aan in de opgave.

Stap 2: Maak een kort schema (tekening) van wat gevraagd wordt.
Werk stap voor stap, want zo denkt een computer ook!



Opgave 2: DwenguinoBlockly!

Stap 3: Open **DwenguinoBlockly** en maak jouw oplossing met de blokken in het deeltje 'Herhaal'.

Stap 4: Download jouw oplossing en dien het in via de uploadzone van het vak!





Wat hebben we geleerd?

Tijdens afgelopen oefeningen leerde je de basis van **computationeel denken** en het hanteren van **programmeerconcepten zoals sequentie, selectie, begrensde herhaling en voorwaardelijke herhaling**. Je leerde dus denken zoals een computer, een algoritme uitwerken en dat omzetten zonder dat je een programmeertaal hoefde te leren. Met zo'n taal, zoals Python, Javascript of C++, kan je natuurlijk véél meer, maar dit is nog maar het begin. We overlopen eens wat we precies hebben geleerd:

Computationeel denken is ...

Een probleem zodanig benaderen, analyseren en opdelen zodat we onze oplossing kunnen laten uitvoeren door een computer.

Waarom?

Steeds meer van onze wereld raakt verbonden met computers en het internet. We kunnen deze rekenkracht gebruiken om problemen op te lossen. Om die oplossingen te kunnen ontwerpen, moet je leren denken zoals een computer.

Hoe?

Als we een probleem willen oplossen met de hulp van een computer, moeten we een algoritme ontwerpen dat de computer kan uitvoeren. Dus vertalen naar een "taal" die de computer begrijpt. Dat doen we als volgt:

o Decompositie

Bij een decompositie wordt een complex probleem opgedeeld in kleinere deelproblemen of stappen.

o Patroonherkenning

Patronen opsporen en begrijpen kan ons helpen om onze oplossing efficiënter te maken! Zo moet je niet telkens hetzelfde uitvinden!

o Abstraheren

We maken gebruik van variabelen en functies om acties of getallen in het geheugen van de computer te bewaren.

o Algoritme ontwerpen

Een algoritme is een stappenplan dat ons naar de juiste oplossing brengt. Een algoritme is schematisch, functioneel en leesbaar.

o Debuggen

Het opmerken van fouten, herkennen en de fout uit het algoritme verwijderen noemen we 'debuggen'.

Programmeerconcepten ...

Programmeerconcepten gebruiken we om onze oplossing, ons algoritme, te maken. Het zijn functies in een programmeertaal die we gebruiken om onze oplossing beter te maken. Je vindt ze terug in nagenoeg alle volwaardige programmeertalen, soms met licht andere notatiewijze of syntax, maar met dezelfde functionaliteit. Je moet dus niet eerst alle computertalen instuderen voordat je deze concepten kan gebruiken in jouw oplossing!

o Sequentie

Wanneer we een algoritme ontwerpen met een schema, met blokjes of met codetaal, zullen computers onze instructies stap voor stap, in sequentie of volgorde, uitvoeren. Ze slaan dus niet zomaar een stap over, maar je kan ze wel pauzeren!

o Selectie

We kunnen code zodanig ontwerpen dat meerdere uitkomsten mogelijk zijn. Hiervoor gebruiken we voorwaarden. Aan de hand van de voorwaarden kan de computer selecteren welke stap er gezet moet worden. Daarvoor gebruiken we een IF-ELSE-functie. IF of ALS aan de voorwaarde wordt voldaan gebeurt het ene, ELSE of anders gebeurt het andere.

o Begrensde herhaling

Bij de begrensde herhaling herkennen en maken we gebruik van een patroon in de code. Bij de begrensde herhaling weten we exact hoeveel keer bepaalde acties herhaald moeten worden. Niet meer, niet minder. Dit doen we aan de hand van een 'for-functie' of 'een teller'.

o Voorwaardelijke herhaling

Hier weten we niet exact hoeveel keer een bepaalde set aan acties herhaald moet worden. We weten alleen wanneer de computer mag stoppen, namelijk wanneer aan een voorwaarde is voldaan. Dit doen we aan de hand van een zolang-functie.



Evaluatie! Heb je alles onder de knie?

Tijdens de voorgaande opdrachten maakte je kennis met het computationeel denken. Je hebt geleerd hoe je een complexe opgave kan analyseren, via programmeerconcepten een efficiënte oplossing kan bedenken en deze schematisch weergeven. Vijf stappen, vier concepten. Om jouw kennis en vaardigheden te evalueren en feedback te geven vind je hieronder enkele uitdagingen!

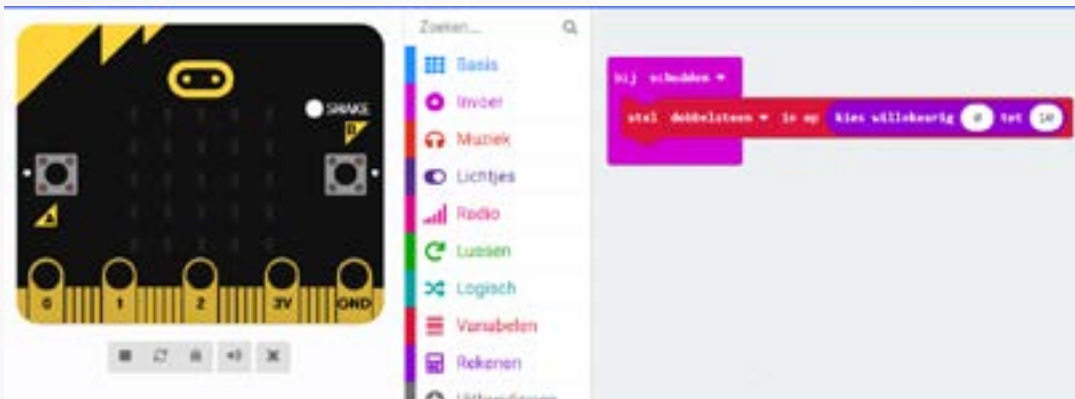


Uitdaging 1: Wat spelen we deze avond?

Een gezelschapsspel natuurlijk! We spelen 'Helden van Andor'. Daarbij proberen vijf helden het koninkrijk van Andor te beschermen van invallende monsters en tegelijk opdrachten uit te voeren doorheen het gehele land. Elke beurt gooit een speler een dobbelsteen en verplaatst zoveel stappen als ogen op de dobbelsteen ... maar jij bent jouw dobbelstenen thuis vergeten!

Programmeer een digitale dobbelsteen in [Makecode:MicroBit](#). Die digitale dobbelsteen toont een cijfer van 1 tot en met 6 via de ledjes van de Micro:Bit.

Download jouw oplossing wanneer je klaar bent!



Uitdaging 2: De tovenaar!

Tijdens het spel kan je kiezen uit verschillende personages, zoals een tovenaar. Die kan met toverspreuken zoals 'vuurbal' en 'ijsstorm' de vijand wegblazen ... als hij de toverspreuk goed kan uitspreken, tenminste. ALS je een 1, 2 of 3 rolt, mislukt de toverspreuk. ANDERS, bij een 4, 5 of 6 kan je een vuurbal werpen naar de vijand!

Programmeer zo een digitale dobbelsteen in [Makecode:MicroBit](#). Die digitale dobbelsteen toont het aantal geworpen ogen via de ledjes van de Micro:Bit én het resultaat van de toverspreuk, namelijk 'mislukt!' of 'gelukt!'

Download jouw oplossing wanneer je klaar bent!





Uitdaging 3: De krijger!

Naast de tovenaar kan je ook kiezen voor de 'krijger'. Die is de allerbeste met zijn zwaard. Tijdens een gevecht met een vijand kan hij drie aanvallen uitvoeren. Bij elke aanval rolt hij 1 dobbelsteen. Wanneer hij een 1 of 2 rolt, mist hij zijn vijand. Bij een 3 of hoger treft hij raak met zijn zwaard en verwordt zo de vijand!

Programmeer zo een digitale dobbelsteen in [Makecode:MicroBit](#). Die digitale dobbelsteen zal een aantal keer na elkaar rollen. Toon telkens, dus per worp, het aantal ogen én het resultaat (= 'gemist' of 'geraakt!'). Zorg dat de speler het aantal ogen en het resultaat kan lezen!

Download jouw oplossing wanneer je klaar bent!

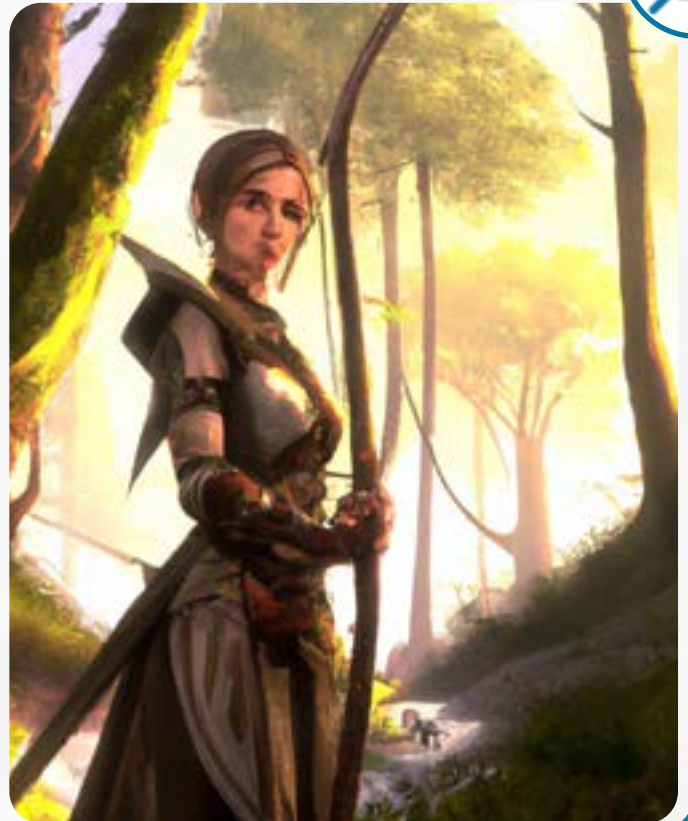


Uitdaging 4: De boogschutter!

Naast de tovenaar en de krijger heeft het spel ook een 'boogschutter'. Dit is een boself die vijanden vanop een afstand kan treffen met pijl en boog. Haar vaardigheid is zo groot dat ze meerdere keren per beurt kan schieten! Op een worp van 3 of meer treft ze haar doel én mag ze nog een pijl schieten. Op een rol van 1 of 2 mist ze haar doel en stopt het schieten.

Programmeer zo een digitale dobbelsteen in [Makecode:MicroBit](#). Die digitale dobbelsteen zal een aantal keer na elkaar rollen. Toon telkens, dus per worp, het aantal ogen én het resultaat (= 'gemist' of 'geraakt!'). Zorg dat de speler het aantal ogen en het resultaat kan lezen!

Download jouw oplossing wanneer je klaar bent!



Indienen!

Wanneer je de 4 uitdagingen op bladzijde 14 en 15 hebt gemaakt en jouw oplossingen hebt gedownload, is het tijd om deze in te dienen. Gebruik daarvoor de passende 'Uploadzone'. Vermeld bij jouw upload jouw naam en eventueel de naam van jouw teamgenoot.

Dit lesproject werd uitgewerkt voor educatief gebruik door Robbe Wulgaert (Sint-Lievenscollege), met oog op het bereiken van de doelen 'computationeel denken' in de eerste graad van het Vlaams Secundair Onderwijs.

Dit lesmateriaal maakt gebruik van DwenguinoBlockly en Micro:Bit. Hiervoor heb je een computer en browser nodig.

Heb je vragen over dit lesproject? Dan kan je contact opnemen via www.aiindeklas.be of www.robbeuwulgaert.be

Bronvermeldingen:

- iconen 'plugged' en 'unplugged': icons-library.com
- Foto Timnit Gebru: [Wikipedia, TechCrunch](https://www.wikipedia.org/wiki/Timnit_Gebru)
- Foto Shoshana Zuboff: [Twitter Shoshana Zuboff](https://twitter.com/ShoshanaZuboff)
- Andere foto's en afbeeldingen: [Robbe Wulgaert](http://RobbeWulgaert.be)